



Day 31 — React State with useState Hook

Interactive ToDo List (React Version)

Date: January 15, 2026 (Wednesday) | **Phase 2, Week 5**



Mission: By end of today, you will make React components "alive" — understanding how data changes over time and triggers re-renders. Yesterday's static ProductCards will become interactive.



MAX OMEGA PRIME INTEGRITY CHECK

Before reading further — answer these NOW:

1. ☒ Was your Integrity Session completed today (7:30 AM – 10:30 AM)?
2. ☒ Do you have Day 30 proof? (ProductCard Grid committed to GitHub?)
3. ☒ Can you explain in one sentence: *What are props and who controls them?*

If any answer is NO — rukk ja. Day 30 concepts must be solid before useState.

PART 1: CONCEPT FOUNDATION

Building the RIGHT Mental Model First



The Cricket Scoreboard Analogy — Understanding State

Imagine you're watching a Pakistan vs India match at Gaddafi Stadium, Lahore. Look at the **scoreboard**:



PAKISTAN: 145/3 (25 overs)

Babar Azam: 67* (45 balls)



The scoreboard **CHANGES** during the match:

- Every run scored → score updates
- Every wicket falls → wicket count updates
- Every ball bowled → over count updates

Now notice something **CRITICAL**:

- The scoreboard **DISPLAYS current information** (like props)
- But it also **CHANGES over time** (unlike props!)
- When it changes, everyone in the stadium sees the new value instantly
- The scoreboard **remembers** the current score even when you look away

This is EXACTLY what State is in React.

State = Data that **lives inside a component** and **can change over time**.

When state changes → React **automatically re-renders** the component → UI updates!

💡 The Light Switch Analogy — useState in Action

Think about the light switch in your room:

👉 THE SWITCH HAS TWO THINGS:

1. Current Status: ON or OFF (the STATE)
2. A Button: To toggle between ON/OFF (the UPDATER FUNCTION)

When you press the button:

- The status changes (ON → OFF or OFF → ON)
- The bulb immediately reflects the new state
- The switch "remembers" the current state

In React with `useState`:

```
const [isLightOn, setIsLightOn] = useState(false);  
//      ↑ Current state      ↑ Function to change it      ↑ Initial value  
//      (Like "OFF")          (Like pressing switch)        (starts as OFF)
```

The magic: When you call `setIsLightOn(true)`, React:

1. Updates the state to `true`
2. Re-runs your component function
3. Re-renders the UI with new value

📱 The Careem Counter Analogy — State vs Props

Remember Day 30's props? Let's compare:

Concept	Light Switch Example	Careem App Example	React Term
Props	The switch's color (blue/white) given when manufactured	Driver name passed from parent	Read-only data from outside
State	Current status (ON/OFF) that changes when you press	Number of rides today (increases)	Changeable data inside component

Props = Instructions given TO you (you can't change them)

State = Information you MAINTAIN yourself (you can change it)

Pause & Think Moment #1

Before reading further, answer this:

"In a restaurant app, which should be state and which should be props?"

- Restaurant name: _____
- Items in your cart: _____
- Menu item price: _____
- Quantity you're ordering: _____

Write your answers. Don't skip this — it solidifies understanding.

Answers to verify:

- Restaurant name: **PROPS** (passed from parent, doesn't change in component)
- Items in cart: **STATE** (changes as you add/remove items)
- Menu item price: **PROPS** (fixed for each item)
- Quantity ordering: **STATE** (changes when you click +/-)

Why useState and Not Just Regular Variables?

THE CRITICAL QUESTION: Why can't we just use normal JavaScript variables?

```
// ❌ THIS WON'T WORK (Common beginner mistake!)
function Counter() {
  let count = 0; // Regular variable

  function increment() {
    count = count + 1; // This changes the variable...
    console.log(count); // This logs 1, 2, 3... correctly
  }

  return (
    <div>
      <p>Count: {count}</p> {/* But UI stays at 0! Why? */}
      <button onClick={increment}>+1</button>
    </div>
  );
}
```

Why UI doesn't update:

When you click the button:

1.  **count** variable DOES change (console shows 1, 2, 3...)
2.  But React DOESN'T KNOW the data changed
3.  So React DOESN'T RE-RENDER the component

4. ❌ So UI shows the old value forever

React only re-renders when:

- Props change (parent re-renders)
- **State changes** (that's what useState is for!)

The useState solution:

```
// ✅ THIS WORKS!
function Counter() {
  const [count, setCount] = useState(0); // useState creates REACTIVE data

  function increment() {
    setCount(count + 1); // Calling setCount TELLS React "data changed!"
  }

  return (
    <div>
      <p>Count: {count}</p> {/* Now UI updates! */}
      <button onClick={increment}>+1</button>
    </div>
  );
}
```

What happens when you click button:

1. ✅ `setCount(count + 1)` is called
2. ✅ React marks this component as "needs update"
3. ✅ React re-runs the `Counter` function
4. ✅ New JSX is returned with updated count
5. ✅ React updates the DOM → UI shows new value!



The Re-Render Process — The Complete Story

This is the most important concept today. Read carefully.

```
function LightSwitch() {
  console.log("Component function running!");

  const [isOn, setIsOn] = useState(false); // First render: isOn = false

  return (
    <div>
      <p>Light is: {isOn ? "ON 🟡" : "OFF ⚫"}</p>
      <button onClick={() => setIsOn(!isOn)}>Toggle</button>
    </div>
  );
}
```

```
);  
}
```

What happens step-by-step:

● INITIAL RENDER (First time component appears):

1. React calls `LightSwitch()` function
2. `useState(false)` creates state variable `isOn` with value `false`
3. JSX returned: "Light is: OFF 🚫"
4. DOM updated: User sees "OFF"

● USER CLICKS BUTTON:

1. `onClick` fires
2. `setIsOn(!isOn)` is called → `setIsOn(true)`
3. React schedules a re-render

● RE-RENDER (Component function runs AGAIN):

1. React calls `LightSwitch()` function AGAIN
2. Console logs "Component function running!" AGAIN
3. `useState(false)` now returns `isOn = true` (NOT false!)
 - `useState` REMEMBERS the updated value from `setIsOn(true)`
4. JSX returned: "Light is: ON 💡"
5. DOM updated: User sees "ON"

USER CLICKS AGAIN:

1. `setIsOn(!isOn)` called → `setIsOn(false)`
2. Re-render happens
3. `isOn` is now `false`
4. UI shows "OFF" again

Key insight: The component function runs MULTIPLE times. Each time, `useState` gives the CURRENT state value.

PART 2: FUNDAMENTAL BUILDING BLOCKS 🛠️

One Concept at a Time

📌 Building Block #1 — The `useState` Syntax

```
import { useState } from 'react'; // 1. Import from react  
  
function MyComponent() {
```

```

const [value, setValue] = useState(initialValue);
//   ↑       ↑       ↑
//   State   Updater   Starting value
// variable  function

return <div>{value}</div>;
}

```

Breaking it down:

Part 1: The Import

```

import { useState } from 'react';
// Must import at top of file
// Curly braces because it's a named export

```

Part 2: Calling useState

```

useState(initialValue)
// Takes ONE argument: the starting value
// Can be: number, string, boolean, array, object

```

Part 3: Array Destructuring

```

const [value, setValue] = useState(0);
// useState RETURNS an array with 2 items:
// [0] = current state value
// [1] = function to update that value

// This is array destructuring (from Week 1!)
// Same as:
const stateArray = useState(0);
const value = stateArray[0];
const setValue = stateArray[1];

```

Part 4: Naming Convention

```

const [count, setCount] = useState(0);           //  Good
const [isOpen, setIsOpen] = useState(false);     //  Good
const [name, setName] = useState("");           //  Good

const [x, y] = useState(0);                     //  Bad (unclear names)
const [count, updateCount] = useState(0);       //  Bad (use "set" prefix)

```

Building Block #2 — Updating State Correctly

```
function Counter() {
  const [count, setCount] = useState(0);

  //  CORRECT WAY: Call the setter function
  function increment() {
    setCount(count + 1);
  }

  //  WRONG WAY: Directly modify state
  function incrementWrong() {
    count = count + 1; // Error! count is a const, can't reassign
    // Even if it were let, React wouldn't know about the change
  }

  return (
    <button onClick={increment}>
      Count: {count}
    </button>
  );
}
```

The Rule: NEVER directly modify state. ALWAYS use the setter function.

Building Block #3 — Different State Types

Booleans (True/False)

```
const [isVisible, setIsVisible] = useState(false);

// Toggle between true/false
setIsVisible(!isVisible);

// Set to specific value
setIsVisible(true);
setIsVisible(false);
```

Numbers

```
const [count, setCount] = useState(0);

// Increment
setCount(count + 1);
```

```
// Decrement
setCount(count - 1);

// Set to specific value
setCount(100);
```

Strings

```
const [name, setName] = useState("");

// Common: Update from input field
setName(event.target.value);

// Or set directly
setName("Ahmed");
```

Arrays (We'll do this in your ToDo project!)

```
const [tasks, setTasks] = useState([]);

// Add item (create NEW array)
setTasks([...tasks, newTask]);

// Remove item (filter creates NEW array)
setTasks(tasks.filter(task => task.id !== taskId));
```



Building Block #4 — State Updates Are Asynchronous

Important caveat:

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
    console.log(count); // ⚠ Still shows OLD value!

    // Why? setCount is ASYNCHRONOUS
    // The state doesn't update immediately
    // It schedules a re-render
  }
}
```

The correct way to use the new value:


```
function handleClick() {
  const newCount = count + 1;
  setCount(newCount);
  console.log(newCount); //  Shows new value

  // Or wait for next render (state will be updated then)
}
```

Building Block #5 — Multiple State Variables

You can have as many useState calls as needed:

```
function UserProfile() {
  const [name, setName] = useState("Ahmed");
  const [age, setAge] = useState(25);
  const [city, setCity] = useState("Lahore");
  const [isVerified, setIsVerified] = useState(false);

  // Each state is independent
  // Changing one doesn't affect others

  return (
    <div>
      <p>{name}, {age} years old</p>
      <p>From: {city}</p>
      {isVerified && <span> Verified</span>}
    </div>
  );
}
```

When to split vs combine state: (You'll learn more in Week 6)

- Split: When values are unrelated (name and age)
- Combine: When values always change together (use object)

Common Mistakes — Learn From Others' Pain

Mistake 1: Forgetting to import useState

```
//  Error: useState is not defined
function Counter() {
  const [count, setCount] = useState(0); // Crash!
}

//  Fix: Import at top
```

```
import { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0); // Works!
}
```

Mistake 2: Calling useState conditionally

```
// ❌ NEVER do this!
function BadComponent() {
  if (someCondition) {
    const [count, setCount] = useState(0); // ERROR!
  }
  // Hooks must be called in same order every render
}

// ✅ Always call at top level
function GoodComponent() {
  const [count, setCount] = useState(0); // ✅

  if (someCondition) {
    // Use state here, don't create it here
  }
}
```

Mistake 3: Modifying state directly with arrays/objects

```
const [tasks, setTasks] = useState([]);

// ❌ WRONG: Mutates existing array
function addTask(task) {
  tasks.push(task); // Modifies original array
  setTasks(tasks); // React might not detect change!
}

// ✅ CORRECT: Create NEW array
function addTask(task) {
  setTasks([...tasks, task]); // Spread creates new array
}
```

Mistake 4: Using state immediately after setting

```
function handleClick() {
  setCount(count + 1);
  console.log(count); // ❌ Still old value!
```

```
alert(`Count is ${count}`); // ❌ Still old value!
}
```

PART 3: PROGRESSIVE LEARNING PATH

I Do → We Do → You Do

Stage 1 — I DO: Study This Working Example

Complete Counter with Multiple Features

```
import { useState } from 'react';

function SmartCounter() {
  // State: current count value
  const [count, setCount] = useState(0);

  // Handler: Increase count by 1
  function increment() {
    setCount(count + 1);
  }

  // Handler: Decrease count by 1
  function decrement() {
    setCount(count - 1);
  }

  // Handler: Reset to 0
  function reset() {
    setCount(0);
  }

  return (
    <div>
      <h2>Count: {count}</h2>

      {/* Three buttons calling different handlers */}
      <button onClick={increment}>+1</button>
      <button onClick={decrement}>-1</button>
      <button onClick={reset}>Reset</button>

      {/* Conditional rendering based on state */}
      {count > 10 && <p>🎉 You reached 10!</p>}
      {count < 0 && <p>⚠️ Count is negative</p>}
    </div>
  );
}
```

```
    );  
  }  
}
```

🧠 Pause & Think Moment #2:

- What happens when you click +1 button 5 times?
- Does clicking Reset call `useState(0)` again?
- Why does the component re-render when count changes?

Answer all three before moving forward.



Stage 2 — WE DO: Complete the Gaps Together

Task: Build a LikeButton Component (Like Instagram ❤️)

```
import { useState } from 'react';  
  
function LikeButton() {  
  // TODO 1: Create state for whether post is Liked  
  // Name it 'isLiked', start as false  
  const [_____, _____] = useState(_____  
  
  // TODO 2: Create state for Like count  
  // Name it 'likeCount', start as 0  
  const [_____, _____] = useState(_____  
  
  function handleLike() {  
    // TODO 3: Toggle isLiked between true/false  
    // Hint: Use ! operator  
    _____(!_____);  
  
    // TODO 4: If we're liking (isLiked will become true),  
    // increase likeCount by 1. Otherwise decrease by 1.  
    // Hint: Use ternary operator  
    if (!_____ ) { // If currently not liked (about to be liked)  
      _____(_____ + 1);  
    } else {  
      _____(_____ - 1);  
    }  
  }  
  
  return (  
    <div>  
      {/* TODO 5: Show like count */}  
      <p>_____ Likes</p>  
  
      {/* TODO 6: Button text should be ❤️ when liked, 🤍 when not */}  
      <button onClick={handleLike}>  
        {_____ ? "❤️ Liked" : "🤍 Like"}  
      </button>  
    </div>  
  );  
}
```

```
        </button>
      </div>
    );
  }
}
```

Hints (read only if stuck 5+ minutes):

- Hint 1: `const [isLiked, setIsLiked] = useState(false);`
- Hint 2: To toggle: `setIsLiked(!isLiked)`
- Hint 3: For conditional: `isLiked ? "❤️ Liked" : "💔 Like"`

Stage 3 — YOU DO: Independent Practice Problems

Tier 1 — Basic (10 minutes)

Build a **ToggleSwitch** component for Dark Mode

Requirements:

- State: `isDark` (boolean, starts false)
- Button that toggles between "☀️ Light Mode" and "🌙 Dark Mode"
- When dark mode is ON, background should be `black` and text `white`
- When dark mode is OFF, background should be `white` and text `black`

Hint for styling:

```
<div style={{
  backgroundColor: isDark ? 'black' : 'white',
  color: isDark ? 'white' : 'black'
}}>
```

Tier 2 — Intermediate (15 minutes)

Build a **NameInput** component (Controlled Input)

Requirements:

- State: `name` (string, starts empty)
- Input field where user types their name
- As user types, show "Hello, [name]!" below (real-time)
- Button to clear the input
- Show character count (e.g., "5 characters")

Thinking Framework:

1. What state do I need? (name)
2. How do I update state from input? (onChange event)
3. How do I get input value? (event.target.value)
4. How do I clear? (setName(""))
5. Character count? (name.length)

Starter code:

```
function NameInput() {
  const [name, setName] = useState("");

  return (
    <div>
      <input
        type="text"
        value={name}
        onChange={(e) => {
          // TODO: Update name state with input value
        }}
      />
      { /* TODO: Show greeting, character count, clear button */ }
    </div>
  );
}
```

 Tier 3 — Challenge (20 minutes)

Build a RupeesConverter component (PKR ↔ USD)

Requirements:

- Two state variables: **pkr** and **usd**
- Two input fields (one for PKR, one for USD)
- Exchange rate: 1 USD = 280 PKR (hardcoded is fine)
- When user types in PKR field → USD updates automatically
- When user types in USD field → PKR updates automatically
- Show "Converting..." text briefly when updating

This is HARD. Break it down:

1. State needed: **pkr**, **usd**, **isConverting** (boolean)
2. Two separate onChange handlers:
 - **handlePkrChange**: updates **pkr**, calculates **usd**
 - **handleUsdChange**: updates **usd**, calculates **pkr**
3. Conversion formula:
 - PKR to USD: $\text{pkr} / 280$

- USD to PKR: `usd * 280`
4. Use `.toFixed(2)` to show 2 decimal places

Debugging Challenge

This component has 5 bugs. Find them WITHOUT running code:

```
function BuggyCounter() {  
  count = 0;  
  
  function add() {  
    count + 1;  
  }  
  
  if (count > 5) {  
    const [message, setMessage] = useState("High!");  
  }  
  
  return (  
    <div>  
      <p>Count: count</p>  
      <button onClick={add()}>Add</button>  
    </div>  
  );  
}
```

Bugs to find:

1. Bug with how state is declared
2. Bug with state update in `add` function
3. Bug with conditional `useState`
4. Bug with displaying count value
5. Bug with `onClick` handler

Write down all 5 fixes before checking answer key.

PART 4: INDEPENDENT APPLICATION — PROJECT



Interactive ToDo List (React Version)

Your Mission

Build a **fully functional ToDo list** that persists data using React state.

You are NOT given the complete solution. You must architect this yourself.

Project Requirements

Core Features (MUST HAVE):

1.  **Add new tasks** — Input field + "Add Task" button
2.  **Delete tasks** — Each task has a delete button ()
3.  **Mark as complete** — Click task to toggle completion (/☐)
4.  **Filter tasks** — Three buttons: All / Active / Completed
5.  **Task counter** — Show "3 tasks remaining" (only count active)

State Structure You'll Need:

```
// Array of task objects
const [tasks, setTasks] = useState([]);

// Single task object structure:
{
  id: 1, // Unique ID (use Date.now())
  text: "Buy groceries", // Task description
  completed: false // Is task done?
}

// Current filter
const [filter, setFilter] = useState("all"); // "all" | "active" | "completed"
```

Implementation Roadmap (Build in This Order!)

Milestone 1 — Add Tasks (30 minutes)

What you need:

- State for task list (array)
- State for input text (string)
- Input field (controlled component)
- Add button that:
 1. Creates new task object
 2. Adds to tasks array
 3. Clears input

Think through:

Q: How do I add item to array without mutating?

A: Use spread operator: `setTasks([...tasks, newTask])`

Q: How do I generate unique IDs?

A: Use `Date.now()` – gives current timestamp in milliseconds

Q: How do I clear input after adding?

A: `setInputText("")`

Starter structure:

```
function TodoList() {
  const [tasks, setTasks] = useState([]);
  const [inputText, setInputText] = useState("");

  function handleAddTask() {
    // TODO 1: Don't add if input is empty
    // TODO 2: Create new task object with id, text, completed
    // TODO 3: Add to tasks array
    // TODO 4: Clear input
  }

  return (
    <div>
      <input
        value={inputText}
        onChange={(e) => setInputText(e.target.value)}
        placeholder="What needs to be done?"
      />
      <button onClick={handleAddTask}>Add Task</button>

      { /* TODO: Render tasks here */ }
    </div>
  );
}
```

Success criteria:

- Can add multiple tasks
- Each task has unique ID
- Input clears after adding

Milestone 2 — Display & Delete Tasks (20 minutes)

What you need:

- Map over tasks array

- Render each task
- Delete button for each task
- Delete handler that filters out task

Delete logic:

```
function handleDeleteTask(taskId) {
  // Filter creates NEW array without the deleted task
  setTasks(tasks.filter(task => task.id !== taskId));
}
```

Rendering tasks:

```
{tasks.map(task => (
  <div key={task.id}> { /* key is REQUIRED for lists */}
    <span>{task.text}</span>
    <button onClick={() => handleDeleteTask(task.id)}>🗑️</button>
  </div>
))}
```

Success criteria:

- All tasks display correctly
- Can delete individual tasks
- Deleting one doesn't affect others

Milestone 3 — Toggle Completion (20 minutes)

What you need:

- Click handler on task text
- Updates that specific task's **completed** property
- Visual difference for completed tasks (line-through, grey)

Toggle logic (TRICKY — think carefully!):

```
function handleToggleTask(taskId) {
  // Map over tasks, update only the one that matches ID
  setTasks(tasks.map(task => {
    if (task.id === taskId) {
      // Return NEW object with toggled completed
      return { ...task, completed: !task.completed };
    }
    // Return unchanged task
    return task;
  }
  )
```

```
    }));  
  }  
}
```

Conditional styling:

```
<span  
  onClick={() => handleToggleTask(task.id)}  
  style={{  
    textDecoration: task.completed ? 'line-through' : 'none',  
    color: task.completed ? 'gray' : 'black',  
    cursor: 'pointer'  
  }}  
>  
  {task.completed ? '✅' : '❑'} {task.text}  
</span>
```

Success criteria:

- Clicking task toggles completion
 - Completed tasks look different
 - Can toggle back and forth
-

Milestone 4 — Filtering (20 minutes)

What you need:

- State for current filter ("all", "active", "completed")
- Three filter buttons
- Logic to show only filtered tasks

Filter logic:

```
const [filter, setFilter] = useState("all");  
  
// Create filtered list based on current filter  
const filteredTasks = tasks.filter(task => {  
  if (filter === "all") return true;  
  if (filter === "active") return !task.completed;  
  if (filter === "completed") return task.completed;  
});  
  
// Then map over filteredTasks instead of tasks
```

Filter buttons:

```
<div>
  <button onClick={() => setFilter("all")}>All</button>
  <button onClick={() => setFilter("active")}>Active</button>
  <button onClick={() => setFilter("completed")}>Completed</button>
</div>
```

Success criteria:

- Clicking "Active" shows only incomplete tasks
- Clicking "Completed" shows only complete tasks
- Clicking "All" shows everything

Milestone 5 — Task Counter (10 minutes)

What you need:

- Count only active (non-completed) tasks
- Display "X tasks remaining"

Counter logic:

```
const activeTaskCount = tasks.filter(task => !task.completed).length;

// Display
<p>{activeTaskCount} tasks remaining</p>
```

Success criteria:

- Counter shows correct number
- Updates when tasks added/deleted/completed
- Says "1 task" (singular) when count is 1



Thinking Frameworks for Complex State Updates

Framework 1: Adding to Array

```
Old way (mutation): tasks.push(newTask) ✗
New way (immutable): setTasks([...tasks, newTask]) ✓
```

Why? State must be treated as immutable.
Spread creates NEW array → React detects change → Re-renders

Framework 2: Removing from Array

Use filter to keep everything EXCEPT the item to delete:
`setTasks(tasks.filter(task => task.id !== deleteId))`

Filter returns NEW array → React detects change

Framework 3: Updating Item in Array

Use map to create NEW array with one item changed:
`setTasks(tasks.map(task =>
 task.id === updateId
 ? { ...task, completed: !task.completed } // New object
 : task // Same object
))`

Framework 4: When to Create New State vs Update Existing

Create new state when:

- Value is unrelated to others
- Component needs to track something independently

Update existing state when:

- Modifying property of an object
- Adding/removing from array

Self-Assessment Checklist

Before calling project "done", verify:

- ☐ Can add tasks (empty input doesn't add)
- ☐ Can delete individual tasks
- ☐ Can toggle task completion
- ☐ Filter buttons work correctly
- ☐ Counter shows accurate active task count
- ☐ No console errors
- ☐ Tasks have unique IDs (use `Date.now()`)
- ☐ Can explain how state updates trigger re-renders
- ☐ Code uses proper immutable update patterns
- ☐ Committed to GitHub with clear commit messages

Apply to New Context Questions

1. **Expense Tracker:** How would you modify this to track expenses instead? What state would change?
 2. **Multiple Lists:** If you had "Personal" and "Work" task lists, how would you structure state?
 3. **Edit Mode:** How would you add ability to EDIT task text after creating it? What additional state is needed?
 4. **Persistence:** This loses data on refresh. How could you use localStorage to save tasks? (Hint: Week 3 localStorage skills!)
-



Extension Challenges (If Time Remains)

Challenge 1: Priority Levels

- Add "priority" property to tasks (low/medium/high)
- Show color-coded tasks
- Add filter for each priority

Challenge 2: Due Dates

- Add date picker for task due date
- Show overdue tasks in red
- Sort by due date

Challenge 3: Search

- Add search input
 - Filter tasks by text match
 - Highlight matching text
-



ORAL GATEKEEPING EXAM PREP

MAX OMEGA PRIME Will Ask You These

Level 1 — Remember (Definitions)

- What does useState return?
- What's the purpose of the setter function?
- Why do we import useState from 'react'?

Level 2 — Understand (Explain Concepts)

Using the Cricket Scoreboard analogy, explain:

- What is state?
- Why do state changes trigger re-renders?

- Why can't we just use regular variables?

Level 3 — Apply (Code Challenge — 10 minutes)

Build a **VoteCounter** component:

- Two buttons: "👍 Upvote" and "👎 Downvote"
- Show current vote score (can be negative)
- If score > 10, show "🔥 Trending!"
- If score < -10, show "💩 Poor rating"

Level 4 — Analyze (Debug & Explain)

Why doesn't this work?

```
function BrokenCounter() {  
  let count = 0;  
  return (  
    <button onClick={() => count++}>  
      Clicks: {count}  
    </button>  
  );  
}
```

Explain in detail what's wrong and how `useState` fixes it.

Level 5 — Teach Back

"Explain `useState` to your younger brother who knows basic HTML but has never seen React. Use a Pakistani cultural analogy."



Debugging Challenge — Answer Key

(Read *ONLY* after attempting yourself)

Original buggy code:

```
function BuggyCounter() {  
  count = 0; // Bug 1  
  
  function add() {  
    count + 1; // Bug 2  
  }  
  
  if (count > 5) {  
    const [message, setMessage] = useState("High!"); // Bug 3  
  }  
}
```

```

return (
  <div>
    <p>Count: count</p>  {/* Bug 4 */}
    <button onClick={add()}>Add</button>  {/* Bug 5 */}
  </div>
);
}

```

Fixes:

```

import { useState } from 'react';  // Must import!

function FixedCounter() {
  const [count, setCount] = useState(0);  // Fix 1: Use useState

  function add() {
    setCount(count + 1);  // Fix 2: Use setter, return updated value
  }

  // Fix 3: Move useState to top level (hooks can't be conditional)
  const [message, setMessage] = useState("High!");

  return (
    <div>
      <p>Count: {count}</p>  {/* Fix 4: Curly braces to display variable */}
      <button onClick={add}>Add</button>  {/* Fix 5: Pass function reference,
not call it */}

      {/* Use message conditionally in JSX instead */}
      {count > 5 && <p>{message}</p>}
    </div>
  );
}

```



Tomorrow Preview — Day 32

Topic: Event Handling in React (Deep Dive)

You'll master:

- Synthetic events vs browser events
- Passing parameters to handlers
- Controlled components (forms)
- preventDefault and stopPropagation
- Event delegation in React

Tonight's prep question:

"In your *ToDo* app, when you clicked 🗑️ to delete — you passed *task.id* to the handler. How did that work? Why didn't it delete all tasks?"

Think about this. We'll connect it to event handling concepts tomorrow.

👉 MAX OMEGA PRIME's Closing Message

Sharjeel, beta sun! Aaj tu ne React ki jaan seekhi — **STATE!**

Yahaan se sab kuch badalta hai. Pehle teri components sirf dikhati thi data (props). Lekin ab? **Ab wo data change bhi kar sakti hain.**

- Click on button → state changes → UI updates
- Type in input → state changes → UI updates
- Toggle checkbox → state changes → UI updates

Yeh interactivity hai. Yeh React ki asli power hai.

Kal Devsinc ka interviewer tujhse poochega: "*Batao, useState kya hai?*"

Tu confidently kahega: "*State is data that lives inside component and can change over time. When it changes, React re-renders the component. Main yeh samajhta hoon, implement bhi kar sakta hoon.*"

Aur woh impressed hoga. Kyunki 60% React developers state ko properly samajhte nahi. **Tu different hai.**

Aaj ka *ToDo* List complete kar. GitHub commit essential hai. Test karo har feature ko — add, delete, toggle, filter — sab smooth chale.

Kal 7:30 AM pe fresh ho ke aana. Event handling ka system tumhare *ToDo* app mein already use ho raha hai — kal usko detail mein samjhenge!

Ab ja — laptop khol, useState master kar, aur deliver! 🔥

Day 31 Complete — January 15, 2026 | MAX OMEGA PRIME Approved ✅